CLOUD OPERATIONS RUNBOOK | OKE-OPS-001

OKE and Pod Troubleshooting

Scenario: unavailable applications, unhealthy worker nodes, and failing Kubernetes pods

1. Purpose

Provide a repeatable path from an OKE alert to evidence, a probable cause, a targeted recovery action, and proof of recovery. The diagnostic sequence below is an operational synthesis; confirm each hypothesis against the affected cluster.

2. Scope

Oracle Cloud Infrastructure Container Engine for Kubernetes (OKE), with managed worker nodes as the primary path. Covers cluster access, scheduling, containers, resources, probes, DNS, Services, OCI networking, load balancers, and persistent volumes. Virtual-node exceptions are marked where relevant.

3. Prerequisites

• Your own working kubeconfig, OCI CLI profile, and a kubectl version compatible with the cluster. OCI IAM and Kubernetes RBAC permissions must cover the required reads and any chosen changes. [1]

• Cluster/compartment OCIDs, region, node pool type, networking mode, namespace, workload owner, and incident/change record.

• Access to application metrics and logs. kubectl top requires a functioning resource metrics API. Node access applies only where supported.

4. Alert trigger conditions

Suggested starting values below are examples, not Oracle defaults. Tune them to workload startup times, baseline behavior, and the service objective.

Signal	Example trigger
User impact	Availability or error-rate objective breached; act immediately.
Pod health	Restarts increase by 3+ in 10 minutes, or desired replicas remain unavailable for 5 minutes.
Scheduling / node	Pod Pending or node NotReady for 5 minutes; investigate sooner for critical workloads.
Resource pressure	CPU >85% of configured limit, or memory >90% of limit for 5 minutes; correlate with latency and OOM events.

Version 1.0 | Prepared 19 October 2026 | Suggested owner: Cloud Operations / OKE Platform Team

5. PROCEDURE | START HERE

Capture evidence and choose a branch

Step 1: confirm the target and establish the failure boundary.

Replace every <placeholder> before running a command. Examples use explicit namespaces to reduce accidental changes to another workload. Cluster-wide reads may require additional RBAC.

```
kubectl config current-context
kubectl get nodes -o wide
kubectl get pods -n <namespace> -o wide
kubectl get events -n <namespace> \
  --sort-by=.metadata.creationTimestamp
kubectl describe pod <pod> -n <namespace>
kubectl logs <pod> -n <namespace> -c <container> \
  --since=15m --timestamps --tail=200
kubectl logs <pod> -n <namespace> -c <container> \
  --previous --timestamps --tail=200
```

Previous logs apply only when a prior container instance is still available. Record timestamps, restart counts, Last State, exit reason, node, ownerReferences, events, and the latest deployment or infrastructure change. Save evidence before restarting. [2,3]

Observed evidence	Follow this path
API timeout / Unauthorized / Forbidden	Cluster access, page 3
Several failing pods share one node	Node health, page 3; then networking, page 8
Pending / image / configuration error	Scheduling and container creation, page 4
CrashLoopBackOff / OOMKilled / slow response	Container and resource investigation, page 5
Running but not Ready / init / termination	Lifecycle and health checks, page 6
Pod healthy; Service or hostname fails	Service and DNS, page 7; OCI path, page 8
PVC Pending / attach or mount failure	Persistent storage, page 9

Reasoning rule: one failing revision suggests an application or configuration change; unrelated applications failing on one node suggest node infrastructure. Treat both as hypotheses and compare with a healthy peer.

5. PROCEDURE | INFRASTRUCTURE

Cluster access and worker nodes

Step 2: distinguish client access from cluster or node failure.

A. API access fails [1]

```
kubectl cluster-info --request-timeout=10s
kubectl auth can-i get pods -n <namespace>
oci ce cluster get --cluster-id <cluster_ocid>
oci ce node-pool get --node-pool-id <node_pool_ocid>
```

Finding	Interpretation and next action
Timeout / no route	Confirm endpoint DNS, chosen public/private endpoint, VPN or Bastion path, route tables, and API endpoint security rules. Compare from a known connected host.
Unauthorized	Check the kubeconfig exec authentication, selected OCI profile, session expiry, and any applicable MFA requirements; use your own credentials.
Forbidden	The API responded but denied the request. Verify the identity and resource/namespace permissions through OCI IAM and Kubernetes RBAC.
OCI work request failed	Open the failed work request in the Console; capture its error and request identifiers before retrying provisioning.

B. Managed node is NotReady or under pressure [4]

```
kubectl describe node <node>
kubectl get pods -A --field-selector spec.nodeName=<node>
kubectl top nodes
kubectl get pods -n kube-system -o wide
```

Read Ready, MemoryPressure, DiskPressure, PIDPressure, taints, and recent node events. Compare the OCI instance lifecycle state and health with Kubernetes status: a running compute instance does not prove a healthy kubelet.

For managed nodes, open the node pool in OCI Console and choose **Troubleshoot Nodes**. Follow the generated SSH or Run Command workflow for Node Doctor and preserve its output/support bundle. Use the current script instructions. [4]

Virtual nodes: Node Doctor and host SSH do not apply. Route host-level failures to the OKE platform owner or Oracle Support. Review feature support before using managed-node procedures. [5]

Technical references: [1] cluster access; [4] Node Doctor; [5] virtual-node comparison. Linked sources are on page 12.

5. PROCEDURE | POD CREATION

Pending, image, and configuration failures

Step 3: use the event message to locate the blocked stage.

```
kubectl describe pod <pod> -n <namespace>
kubectl get resourcequota,limitrange -n <namespace>
kubectl get pvc -n <namespace>
kubectl get nodes --show-labels
```

A. Pending / FailedScheduling [2]

Event evidence	Reasoned response
Insufficient cpu / memory	Compare requests with node allocatable resources and allocated requests. Low current usage does not guarantee schedulable capacity. Add eligible capacity or correct an oversized request.
Untolerated taint / affinity mismatch	Compare pod selectors, required affinity, tolerations, and topology constraints with eligible nodes. Correct the intended placement rule; preserve isolation requirements.
Unbound volume claim	Inspect PVC events and StorageClass binding mode. Continue on page 9.
Quota / admission denial	Inspect the owning Deployment or ReplicaSet events too: pod creation may have been rejected before a pod exists. Correct quota, policy, or manifest validation.

B. ErrImagePull / ImagePullBackOff [6]

The image has not been pulled successfully. Inspect the complete event text: denied/unauthorized points to registry credentials or permission; not found points to repository/tag/digest; timeout points to DNS or network reachability. Verify image architecture for the worker shape.

```
kubectl get pod <pod> -n <namespace> -o yaml
kubectl get secret <pull-secret> -n <namespace> \
  -o jsonpath='{.type}{"\n"}'
```

Check imagePullSecrets references in the pod or ServiceAccount and that the secret exists in the same namespace. For OCIR, validate registry hostname, tenancy/repository path, and the credential mechanism used by the deployment.

C. CreateContainerConfigError / ContainerCreating

A missing Secret or ConfigMap is a configuration issue; FailedMount is a storage branch; FailedCreatePodSandBox is a CNI/IP branch. Use describe output to identify the named object or operation. Repair the workload source, then redeploy. Do not print Secret values into incident logs.

Technical references: [2] debug pods; [6] private registry image pulls. Linked sources are on page 12.

5. PROCEDURE | CONTAINER HEALTH

Crash loops, memory, and CPU pressure

Step 4: identify what terminated the container or slowed it down.

A. CrashLoopBackOff is a retry state [3]

```
kubectl describe pod <pod> -n <namespace>
kubectl logs <pod> -n <namespace> -c <container> \
  --previous --timestamps --tail=200
kubectl get deployment <deployment> -n <namespace> -o yaml
kubectl rollout history deployment/<deployment> -n <namespace>
```

Match the termination time with logs and events. Application exceptions suggest code, startup arguments, configuration, or dependency failure. Probe failures suggest page 6. A clean exit in a long-running service can mean the command completed instead of starting the server. Check each failing container separately, including sidecars.

B. OOMKilled and evictions [7]

```
kubectl top pod <pod> -n <namespace> --containers
kubectl top nodes
kubectl describe node <node>
```

Use the OOMKilled reason plus historical memory and node evidence. Exit code 137 alone does not prove a memory-limit breach. A limit-related OOM suggests a memory spike, leak, or limit below the actual requirement; a pod marked Evicted requires reading the eviction message and node pressure.

If evidence supports resizing, confirm eligible node capacity and adjust requests/limits in the workload source. For managed nodes, choose requests that reflect expected use. Virtual-node resource settings have additional constraints; consult [5]. Investigate recurring growth after recovery.

C. High CPU or latency [7]

Compare application demand, CPU usage, configured limits, throttling metrics where available, and downstream latency. A top snapshot can miss short spikes. Scale replicas only if the application supports it and scheduling capacity exists; check HPA status and its metrics first.

```
kubectl get hpa -n <namespace>
kubectl describe hpa <hpa> -n <namespace>
```

Choose the smallest supported action: fix configuration or a failing dependency, roll back a bad revision, or resize/scale for demonstrated demand. A restart can remove evidence and only temporarily hide the cause.

Technical references: [3] debug running pods; [5] virtual nodes; [7] resource management; [15] Deployments. Linked sources are on page 12.

5. PROCEDURE | POD LIFECYCLE

Readiness, init, and termination

Step 5: inspect the transition that is not completing.

A. Running but not Ready [8]

Running describes lifecycle state, not successful service delivery. Review container Ready conditions and probe failure events. Readiness controls traffic eligibility; liveness can restart a container; startup probes allow initialization before the other probes take over.

```
kubectl describe pod <pod> -n <namespace>
kubectl get deployment <deployment> -n <namespace> -o yaml
kubectl get endpointslice -n <namespace> \
  -l kubernetes.io/service-name=<service> -o yaml
```

Check probe path, port, scheme, timeouts, and whether the application listens on the expected interface. Compare actual cold-start time with probe settings. Fix the endpoint or timing based on measurements. Investigate dependency failure rather than simply disabling readiness or liveness.

B. Init container does not complete [9]

```
kubectl logs <pod> -n <namespace> -c <init-container> \
  --timestamps --tail=200
kubectl logs <pod> -n <namespace> -c <init-container> \
  --previous --timestamps --tail=200
```

Use initContainerStatuses to identify the blocking init container. Verify its dependency endpoint, credentials references, mount permissions, and migration status. A blocked init container can prevent the main application from starting. Do not rerun a non-idempotent migration until its actual outcome is known.

C. Terminating longer than expected [10]

```
kubectl get pod <pod> -n <namespace> -o yaml
kubectl describe pod <pod> -n <namespace>
kubectl get volumeattachments
```

Check deletion timestamp, finalizers, node reachability, shutdown hooks, and CSI detach progress. A finalizer represents pending cleanup; repair or recover its controller first. Force deletion does not establish that the old process stopped, especially on an unreachable node.

Stateful workloads: verify the previous writer is stopped or fenced before recovery on another node. Forced deletion or attachment changes can risk duplicate writers and data corruption. Escalate unresolved storage ownership to the storage/platform team.

Technical references: [8] probes; [9] init containers; [10] finalizers. Linked sources are on page 12.

5. PROCEDURE | APPLICATION CONNECTIVITY

Services and DNS

Step 6: test one network layer at a time.

```
kubectl get service <service> -n <namespace> -o yaml
kubectl get pods -n <namespace> --show-labels
kubectl get endpointslice -n <namespace> \
  -l kubernetes.io/service-name=<service> -o yaml
kubectl get networkpolicy -n <namespace>
```

A. Trace application -> pod IP -> Service -> DNS [11,12]

Use a healthy diagnostic client in the affected namespace and network-policy context. The following managed-node examples require the named container to contain nslookup and curl. If tools are missing, use an approved diagnostic image through your platform process; exec and debug capabilities differ on virtual nodes. [5]

```
kubectl exec <client-pod> -n <namespace> -c <container> -- \
  nslookup <service>.<namespace>.svc.cluster.local
kubectl exec <client-pod> -n <namespace> -c <container> -- \
  curl -sv --connect-timeout 5 http://<pod-ip>:<target-port>/
kubectl exec <client-pod> -n <namespace> -c <container> -- \
  curl -sv --connect-timeout 5 http://<cluster-ip>:<service-port>/
```

Examples assume HTTP and the default cluster DNS suffix; substitute the actual protocol, port, and DNS domain. Test an application-defined lightweight endpoint.

Result	What to investigate next
No ready endpoints	Check the Service selector, pod labels, readiness, and EndpointSlice ready condition.
Pod IP fails	Check app listener, target port, policy, and CNI path. Compare same-node and cross-node traffic.
Pod IP works; ClusterIP fails	Check Service port/targetPort and the configured Service data plane (kube-proxy or its replacement).
ClusterIP works; name fails	Check client resolv.conf, CoreDNS pods/logs, DNS Service endpoints, and UDP/TCP 53 reachability.

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100
```

Technical references: [5] virtual nodes; [11] debug Services; [12] DNS debugging. Linked sources are on page 12.

5. PROCEDURE | OCI NETWORKING

CNI, OCI paths, and load balancers

Step 7: correlate Kubernetes symptoms with the OCI network path.

A. Sandbox creation fails or cross-node traffic breaks [13]

Identify the actual CNI/networking mode first. With OCI VCN-native networking, pods consume addresses from configured pod subnets. Check usable IP capacity, VNIC/address allocation errors, pod limits, and node-pool pod networking configuration. Use the network rules for that mode rather than applying overlay assumptions.

```
kubectl get pods -n kube-system -o wide
kubectl get daemonsets -n kube-system
kubectl describe pod <pod> -n <namespace>
```

Find the actual CNI components before reading their logs; names vary. Compare failing and healthy nodes. In OCI Console, inspect the relevant pod/worker subnet routes, NSGs/security lists, and destination path. For external dependencies, verify the required NAT, service gateway, or other routed path and DNS reachability.

B. LoadBalancer address remains pending [14]

```
kubectl describe service <service> -n <namespace>
kubectl get service <service> -n <namespace> -o yaml
kubectl get ingress -n <namespace>
```

Read provisioning events. Check the selected LB/NLB type, annotation names and values, target subnet, permissions, available IPs, and service limits. Treat an Ingress as a separate resource requiring a functioning controller and matching class. Trace the error to the OCI work request where one is available.

C. Address exists but requests fail [14]

Compare a direct Service test with the external request. Inspect OCI backend health, listener port/protocol, health-check path/port, TLS certificate and hostname, and security rules for the actual backend path. Determine whether backends target nodes or pods before selecting the relevant rules.

Reasoning: healthy direct pod and Service tests narrow the failure toward ingress or the external path. Unhealthy backends require proving application readiness and health-check reachability. A blanket network-rule change can obscure the original fault.

D. Virtual-node application checks [16]

Use Oracle's documented kubectl proxy workflow for HTTP application access on virtual nodes in place of port-forward. Access through an API proxy does not prove that the client-to-load-balancer path works.

Technical references: [13] OCI VCN-native CNI; [14] LB/NLB configuration; [16] virtual-node proxy. Linked sources are on page 12.

5. PROCEDURE | PERSISTENT STORAGE

PVC, attach, and mount failures

Step 8: separate provisioning, scheduling, attachment, and filesystem access.

```
kubectl get pvc -n <namespace>
kubectl describe pvc <pvc> -n <namespace>
kubectl get storageclass <storageclass> -o yaml
kubectl get pv <pv> -o yaml
kubectl get volumeattachments
kubectl describe pod <pod> -n <namespace>
```

Evidence	Interpretation and next check
PVC Pending; waiting for first consumer	With WaitForFirstConsumer, delayed binding can be expected. Inspect scheduling events on the consuming pod before treating the PVC as a provisioner failure. [17]
ProvisioningFailed	Read the exact CSI error. Validate StorageClass parameters, CSI component health, required OCI permissions, volume limits, and encryption-key access where used. [17]
Volume node affinity conflict	Compare PV topology/availability domain with eligible node placement. Correct the placement or use a planned storage migration.
FailedAttachVolume / Multi-Attach	Inspect current attachments, access mode, old pod/node state, and CSI events. Establish that an old writer has stopped before moving a stateful workload.
FailedMount / permission denied	Check filesystem/mount options, application user/group, and volume permissions. For OCI File Storage, inspect mount target routing, export rules, and required network access. [18]

Choose recovery based on the blocked stage

For provisioning errors, repair the StorageClass, permission, limit, or key-access problem in the managed configuration. For a placement conflict, provide an eligible node in the volume topology. For attach/mount errors, coordinate with the storage owner and preserve events and volume identifiers.

Data protection: do not delete a PVC/PV, strip finalizers, force-detach a volume, or recreate a StatefulSet as a generic troubleshooting step. Inspect reclaim policy and the recovery plan first. Storage recovery must account for write ownership and application consistency.

After recovery, confirm the claim is Bound, the pod mounts it without repeated events, and the application can perform an approved read/write check. A Bound claim by itself does not prove a usable filesystem or correct application access.

Technical references: [17] OCI Block Volume PVCs; [18] OCI File Storage PVCs. Linked sources are on page 12.

5. PROCEDURE | RECOVERY ACTIONS

Apply one targeted change

Step 9: choose a recovery action only after the evidence supports it.

The examples below change cluster state. Use your incident/change process, verify the context and owner, and record the previous configuration. Check available replicas, spare capacity, PodDisruptionBudgets (PDBs), and storage behavior before any disruptive operation.

A. Roll back a confirmed bad Deployment revision [15]

```
kubectl rollout history deployment/<deployment> -n <namespace>
kubectl rollout undo deployment/<deployment> -n <namespace> \
  --to-revision=<known-good-revision>
kubectl rollout status deployment/<deployment> -n <namespace> \
  --timeout=300s
```

Use when the incident aligns with a release and the selected revision is compatible with current data/configuration. Deployment rollback restores the pod template; it does not roll back an external database, independently changed ConfigMaps, or Secrets. Reconcile the deployment source/GitOps state.

B. Restart after correcting a transient/configuration fault [15]

```
kubectl rollout restart deployment/<deployment> -n <namespace>
kubectl rollout status deployment/<deployment> -n <namespace> \
  --timeout=300s
```

Use only when replacement containers can start successfully. A rollout may reduce availability for a single replica or constrained cluster. If the failure repeats, stop repeating restarts and revisit the evidence. Handle StatefulSets and Jobs with workload-specific recovery procedures.

C. Isolate and recover a confirmed unhealthy managed node [19]

```
kubectl get pdb -A
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --timeout=10m
# After repair and validation, if the same node is retained:
kubectl uncordon <node>
```

Cordon blocks normal new placement; it does not move existing pods. Drain uses eviction and normally respects PDBs; it may block for local data or unmanaged pods. Resolve the blocking condition without adding force, eviction-bypass, or data-deletion flags as a routine shortcut. Confirm replacement capacity before repair/replacement through OKE.

Stop condition: if a change creates new unavailable replicas, fails scheduling, or worsens user impact, pause further changes and restore the recorded configuration where safe. Use the escalation path on page 11.

Technical references: [15] Deployments; [19] safely drain / drain command. Linked sources are on page 12.

6-9. VALIDATION AND OWNERSHIP

Verify recovery and close the incident

Step 10: confirm service behavior, then record the cause and follow-up.

6. Post-remediation validation

```
kubectl get pods -n <namespace> -o wide
kubectl rollout status deployment/<deployment> -n <namespace> \
  --timeout=300s
kubectl get endpointslice -n <namespace> \
  -l kubernetes.io/service-name=<service>
kubectl get events -n <namespace> \
  --sort-by=.metadata.creationTimestamp
```

• Desired replicas are available; new containers are Ready; restart counts are stable across samples. Old failed or evicted pod objects may remain as historical evidence.

• Relevant nodes are Ready without persistent pressure. Storage is mounted and verified; endpoints and OCI backend health match the intended traffic path.

• A representative request succeeds from the real client path; latency/error rate and resource metrics return to baseline. Observe for at least 15 minutes as a suggested starting point, longer for intermittent failures.

7. Escalation criteria and evidence

When / route	Provide
Immediate: critical outage, multiple nodes or namespaces, suspected data risk -> incident lead and OKE platform team	Impact, first/last observed times with timezone, recent changes, cluster OCID/region/version, node pool type, CNI mode.
App-specific failure persists after one targeted recovery -> application owner	Namespace, pod/container/node, owner/revision/image digest, sanitized logs, termination reasons, events, and healthy-peer comparison.
OCI/node/CNI/storage issue persists or work request fails -> platform/network/storage team; Oracle Support as needed	Node/volume/LB OCIDs, OCI work/request IDs, exact errors, relevant health metrics, Node Doctor bundle for managed nodes, changes tried and their outcomes.

8. Risks and future improvements

Restarts/drains can interrupt traffic; stateful recovery can affect data integrity. Preserve logs and sanitize credentials/customer data before sharing. Reduce recurrence through tested probes, right-sized resources, capacity/IP monitoring, disruption budgets, and alerts tied to service objectives.

9. Runbook owner and closure record

Suggested owner: Cloud Operations / OKE Platform Team. Before adoption, assign the named owner, escalation contacts, and environment thresholds. Review after incidents and cluster upgrades. Record: incident ID, cause or open hypothesis, evidence, action, validation window, follow-up owner, and next review date.

REFERENCE LIBRARY

Official technical references

Oracle Cloud Infrastructure and Kubernetes documentation | Checked 19 October 2026

[1] OKE: Setting Up Cluster Access

[2] Kubernetes: Debug Pods

[3] Kubernetes: Debug Running Pods

[4] OKE: Node Doctor troubleshooting

[5] OKE: Comparing Virtual Nodes with Managed Nodes

[6] Kubernetes: Pull an Image from a Private Registry

[7] Kubernetes: Resource Management for Pods and Containers

[8] Kubernetes: Configure Liveness, Readiness and Startup Probes

[9] Kubernetes: Debug Init Containers

[10] Kubernetes: Finalizers

[11] Kubernetes: Debug Services

[12] Kubernetes: Debugging DNS Resolution

[13] OKE: OCI VCN-Native Pod Networking CNI

[14] OKE: Configuring Load Balancers and Network Load Balancers

[15] Kubernetes: Deployments

[16] OKE: Virtual-node troubleshooting with kubectl proxy

[17] OKE: Provisioning PVCs on Block Volume

[18] OKE: Provisioning PVCs on File Storage

[19] Kubernetes: kubectl drain command reference

[20] OKE: Known Issues (check against the affected version)

Reference and usage notes

The supplied kb-file.pdf informed the runbook structure only. Its VM restart procedure and internal classification label were not carried over. Thresholds, observation windows, and diagnostic inferences here are suggested operating choices. Commands are examples reviewed against documentation; they have not been executed against a live cluster. Check OKE known issues and supported features for your cluster version before applying a fix. [20]